

AI Application

1주차: 강의 소개 및 AI 개요

세상을 떠난 지 11년 된 사람의 목소리로, 어떻게 '새로운' 방송이 만들어질까?

2025년 12월 14일, 유튜브 채널 'Ghoststation: the Next' 공개 — 오늘 수업의 첫 질문입니다

사례: 고스트스테이션: 더 넥스트 (Ghoststation: the Next)

- 원작: 고(故) 신해철이 2001년부터 11년간 진행한 심야 라디오 '고스트스테이션'
- 2014년 그가 세상을 떠난 뒤 방송도 멈춤
- 2025년 12월 14일: 넥스트유나이티드(대표: 유족 윤원희)가 AI로 재해석한 새 방송을 유튜브 채널 'Ghoststation: the Next'로 공개
- 제작 방식: 고인이 남긴 음성·텍스트·이미지 데이터를 여러 AI 도구로 학습·조합
 - 대본 구성 + 음성 복원 + 비주얼 아트를 AI로 제작
- 제작진의 입장: 고인의 '부활'이 아니라, 그가 남긴 질문과 가치를 잇는 문화적 실험
 - AI 신해철은 매회 "나는 신해철이 아니다"라고 밝힘

**"나는 신해철이 아니다 —
나는 신해철의 '확률'이다"**

고스트스테이션: 더 넥스트, AI 신해철의 오프닝 멘트 취지 (뉴스 보도 기준 취지 인용 — 정확한 워딩은 방송마다 다를 수 있음)

"확률이다"라니? — 이 문장을 기술적으로 풀면

- AI 신해철 = 고인이 남긴 음성·텍스트·이미지 데이터로부터 학습된 확률 분포(probability distribution) 그 자체
 - 말투·어휘·목소리 톤·생각의 패턴이 데이터 속에 분포로 남아 있음
- 매회 방송 = 그 분포에서 뽑은 새로운 샘플(sampling)
 - 과거 발언의 '재생'이 아니라, "그 사람이라면 이렇게 말할 확률이 높은" 새로운 문장
- 그래서 '부활'이 아니라 '확률'이라는 표현이 기술적으로 정확함
- 이것이 이번 학기 전체를 관통하는 관점: "학습 = 함수/분포 찾기"
 - 오늘 PART 3에서 이 관점을 정면으로 다루고, 12주차 생성형 AI에서 완성

※ 지금은 다 이해 못 해도 좋습니다 — 15주 뒤에는 이 문장이 당연해집니다

같은 원리, 다른 무대 — Beethoven X: The AI Project

- 베토벤은 교향곡 10번의 스케치 단편들만 남기고 1827년 사망
- 2021년 10월, 탄생 250주년 프로젝트 "Beethoven X: The AI Project" — AI가 완성본을 만들어 독일 본(Bonn)에서 초연
- 방법:
 - ① AI가 베토벤 전 작품을 학습 → '베토벤 스타일'의 분포를 획득
 - ② 남은 스케치를 조건(condition)으로 뒷부분을 생성 — 조건부 생성 $p(\text{다음 악보} \mid \text{스케치, 베토벤 스타일})$
 - ③ 음악학자·작곡가 팀이 후보들을 선별·오케스트레이션 (human-in-the-loop)
- 고스트스테이션과 완전히 같은 원리 — 목소리·문장이든 악보든, "그 사람의 데이터 분포를 학습하고, 남긴 단서를 조건으로 샘플링"

Beethoven X가 남긴 토론 거리

평론가들의 상반된 반응

- "베토벤답게 들린다" — 스타일 재현은 성공적
- "베토벤의 논리(전개)가 없다" — 악상 전개 of 필연성 부재
- → 분포를 흉내내는 것 vs 이해하는 것의 차이?
- PART 3에서 다룬 블랙박스·UAT 한계("근사 가능 ≠ 이해")와 이어지는 질문

인간의 선별이 필수였다

- AI는 후보를 '생성'했을 뿐, 무엇이 좋은지는 사람이 판단
- 선별(curation)·오케스트레이션 없이는 작품이 되지 못함
- → AI 출력물에는 평가(evaluation)와 검증이 반드시 따라야 한다
- 이 수업에서 '평가'를 반복 강조하는 이유의 첫 번째 예고편

잠깐, 그래도 되는 걸까? — 윤리라는 질문

- 고인의 인격권: 세상을 떠난 사람의 목소리·이미지를 누가, 어디까지 쓸 수 있나
- 딥페이크(deepfake) 악용 가능성: 같은 기술이 사기·명예훼손에 쓰일 수 있음
- 이 사례의 안전장치:
 - 유족이 직접 주도 (넥스트유나이티드 대표 = 유족)
 - 왜곡 방지를 위한 윤리 가이드라인 운영
 - 매회 "나는 신해철이 아니다"라고 스스로 밝힘 — AI임을 투명하게 고지
- 기술이 '할 수 있는 것'과 '해도 되는 것'은 다른 문제 → 12주차(생성형 AI 윤리)에서 재론

교수 및 과목 소개

- **담당: AI Application 담당 교수 (스마트모빌리티학과)**
 - 연구 분야: 모빌리티 데이터 분석, 자율주행 인지(Perception)
 - 연락: Mattermost 채널 우선, 이메일은 24시간 내 회신 원칙
- **오피스아워(Office Hour): 주 2회, 시간은 Mattermost 채널 공지 참조**
 - 사전 예약 없이 방문 가능, 과제·프로젝트 상담 환영
- **질문 문화: "모르는 것"이 아니라 "아직 안 배운 것"**
 - 수업 중 질문은 언제든지 환영 — 손들기 or 채팅

※ 2·3·4학년 혼합 수업 — 서로 수준이 다른 것이 정상입니다

이 과목은 무엇을 배우나?

- AI를 "만드는 법"이 아니라 "쓰는 법"부터 시작하는 응용 과목
 - 수학 증명보다 코드 실행과 결과 해석에 집중
- Python → 머신러닝(ML) → 딥러닝(DL) → 생성형 AI까지 한 학기 완주
- 모든 예제는 스마트모빌리티 도메인: 교통량, 주행 영상, 차량 센서
- 선수 지식: 프로그래밍 경험이 없어도 2주차부터 따라올 수 있게 설계
 - 2학년: 기초부터 차근차근 / 3·4학년: 프로젝트에서 심화
- 최종 목표: 나만의 모빌리티 AI 미니 프로젝트 완성 (14~15주차)

오늘의 학습 목표

- 강의 운영 방식(시연 중심, 자동채점 과제, 피어 튜터링)을 이해한다
- $AI \supset ML \supset DL$ 의 포함 관계와 각각의 정의를 설명할 수 있다
- AI 역사의 주요 전환점 5가지를 순서대로 나열할 수 있다
- 스마트모빌리티 분야의 AI 적용 사례를 5가지 이상 제시할 수 있다
- AI 개발 파이프라인 4단계(데이터→모델→평가→배포)를 설명할 수 있다
- Google Colab에서 노트북을 열고 셀을 실행할 수 있다

오늘의 목차

- PART 1. 강의 운영 안내 — 이 수업은 이렇게 굴러갑니다
- PART 2. AI·머신러닝·딥러닝 — 용어부터 정확히
- PART 3. AI를 보는 관점 — 결국 함수 찾기
- PART 4. AI의 역사 — 튜링 테스트에서 생성형 AI까지
- PART 5. 스마트모빌리티 × AI — 우리 분야의 적용 사례
- PART 6. AI 개발 파이프라인과 Google Colab
 - 마지막 30분: Colab 시연(Demo) — 직접 따라 하기

PART 1

강의 운영 안내

시연 중심 수업 · 자동채점 과제 · 3인 1조 피어 튜터링

수업 방식: 시연(Demo) 중심

- 매주 3시간 = 이론 강의 + 라이브 시연 + 직접 실습
- 교수가 Colab에서 코드를 직접 실행하며 설명 (라이브 코딩)
 - 에러가 나는 과정도 그대로 보여줌 — 에러 읽는 법이 실력
- 학생은 같은 노트북을 열어 실시간으로 따라 실행
- 조교(TA)가 없는 수업 → 막히면 조원에게 먼저, 그다음 교수에게
- 노트북(laptop) 필수 지참 — 배터리 충전 확인!

주간 3시간 수업 구성

시간	활동	내용
0:00 ~ 0:50	이론 강의	이번 주 개념 설명 (슬라이드)
0:50 ~ 1:00	휴식	-
1:00 ~ 1:50	라이브 시연	교수 시연 + 학생 따라하기 (Colab)
1:50 ~ 2:00	휴식	-
2:00 ~ 2:50	조별 실습	빈칸 채우기 실습 + 피어 튜터링
2:50 ~ 3:00	정리	핵심 요약, 과제 안내, 질의응답

※ 주차별로 이론/실습 비중은 조정될 수 있음

과제: 빈칸 채우기 노트북 + 자동채점

- **매주 과제 = 수업 노트북의 확장판, 핵심 코드가 빈칸(____)으로 제공**
 - 예: `model.__(X_train, y_train) → fit` 을 채우기
- **노트북 안의 채점 셀(assert 기반)을 실행하면 즉시 정답 확인**
 - 제출 전에 스스로 몇 점인지 알 수 있음 — 만점 제출이 목표
- **제출(기본): ① 본인 GitHub 저장소 push → ② e-Campus 제출 → ③ 이메일로 알림**
 - GitHub 사용이 어려우면 .ipynb를 이메일에 첨부해 제출해도 인정 (e-Campus 제출은 공통 필수)
 - 메일 제목: [AI응용] 과제N_학번_이름 / 본문에 GitHub 저장소 링크 포함
 - 마감: 다음 수업 전날 23:59
- **지각 제출: 하루당 10% 감점, 3일 이후 미인정**
- **표절: 채점 셀 조작·코드 복붙 적발 시 해당 과제 0점**

자동채점은 이렇게 동작합니다

```
# [실습] 빈칸을 채우세요: 리스트에서 평균 속도를 계산
speeds = [42, 55, 38, 60, 47]          # 차량 5대의 속도 (km/h)
avg_speed = sum(speeds) / ____ (speeds) # <-- 빈칸!

# ---- 채점 셀 (수정 금지) ----
def grade():
    score = 0
    try:
        assert abs(avg_speed - 48.4) < 1e-6
        score += 10
        print("[PASS] 평균 속도 계산 (+10점)")
    except (AssertionError, NameError):
        print("[FAIL] 평균 속도 계산 - 다시 시도하세요")
    print(f"현재 점수: {score}/10")

grade()
```

※ 빈칸에 len 을 채우고 실행하면 즉시 PASS/FAIL이 표시됩니다

3인 1조 피어 튜터링(Peer Tutoring)

왜 조를 짜나요?

- 조교가 없는 30명 수업 — 서로가 서로의 조교
- 설명해 보는 것이 최고의 복습 (러닝 피라미드)
- 2·3·4학년 혼합 → 학년 섞어서 편성
- 실습 중 막히면: ① 조원 → ② 다른 조 → ③ 교수
- 프로젝트(14~15주)도 같은 조로 진행

운영 규칙

- 10개 조 × 3인, 2주차에 확정 공지
- 매주 실습 시간에 조별로 모여 앓기
- "튜터" 역할은 매주 돌아가며 담당
- 튜터는 답을 알려주지 않고 힌트만 제공
- 조 기여도는 프로젝트 동료평가에 반영

평가 비율

항목	비율	내용
중간고사	25%	8주차, 이론 + 코드 빈칸 문제
기말고사	25%	16주차, 이론 + 코드 빈칸 문제
프로젝트	25%	3인 1조 모빌리티 AI 프로젝트 (발표 + 보고서 + 동료평가)
과제	15%	주간 빈칸 채우기 노트북 (자동채점)
출석	10%	전자출결, 지각 3회 = 결석 1회

※ F 기준: 결석 1/3 초과 또는 프로젝트 무단 불참

수강 준비물

- 노트북(laptop): 사양 무관 — 브라우저만 돌아가면 OK
 - 모든 실습은 Google Colab 클라우드에서 실행 (GPU 무료 제공)
- Google 계정: Colab과 Drive 사용에 필수, 오늘 중 확인
- 소통 채널: 강의자료는 Google Drive·GitHub, 과제는 GitHub + e-Campus 제출 후 이메일 알림, 공지·질문은 Mattermost 채널
- 설치 프로그램: 없음! (로컬 Python 설치는 선택 사항)
- 교재: 별도 구매 없음 — 매주 슬라이드 + 노트북 제공

**코드는 눈으로 읽는 것이 아니라
손으로 실행하는 것입니다**

이 수업에서 여러분은 매주 최소 하나의 노트북을 직접 실행하게 됩니다

■ 중간 점검 퀴즈 #1

Q. 이 수업의 평가 항목 중 비중이 가장 큰 것을 모두 고르면?

- ① 중간고사 (25%)
- ② 기말고사 (25%)
- ③ 프로젝트 (25%)
- ④ ①·②·③ 모두 같다

▶ 정답: ④ — 중간·기말·프로젝트가 각 25%로 동물, 과제 15% + 출석 10%까지 합쳐 100%

PART 2

AI · 머신러닝 · 딥러닝

용어를 정확히 구분하는 것이 첫걸음

인공지능(AI, Artificial Intelligence)이란?

- 정의: 인간의 지적 능력(인지, 추론, 학습, 판단)을 기계로 구현하는 기술
 - 1956년 다트머스 회의(Dartmouth Conference)에서 용어 탄생
- 넓은 개념: 규칙 기반 시스템부터 ChatGPT까지 모두 AI
- 예: 내비게이션 경로 탐색, 스팸 필터, 자율주행, 음성 비서
- 핵심 질문: "기계가 지능적으로 행동하는가?" (방법은 불문)
 - 체스 프로그램은 if-else 덩어리여도 AI라고 부를 수 있음

AI ⊃ 머신러닝(ML) ⊃ 딥러닝(DL)

딥러닝은 머신러닝의 한 방법이고, 머신러닝은 AI를 구현하는 한 방법입니다

머신러닝(ML): 규칙을 짜지 말고 데이터로 학습

전통 프로그래밍

- 사람이 규칙(rule)을 직접 코딩
- 입력 + 규칙 → 출력
- 예: "속도 > 제한속도이면 경고"
- 규칙이 명확한 문제에 적합
- 복잡한 패턴(보행자 인식 등)은 규칙 작성이 사실상 불가능

머신러닝

- 입력 + 출력(정답) → 규칙을 기계가 학습
- 데이터에서 패턴을 스스로 발견
- 예: 보행자 사진 1만 장 → 보행자 인식 규칙 학습
- 규칙을 말로 설명하기 어려운 문제에 강함
- 데이터의 양과 질이 성능을 좌우

머신러닝의 고전적 정의 (Tom Mitchell, 1997)

- "작업 T의 성능 P가 경험 E를 통해 개선되면, 그 프로그램은 학습한다"
- 스마트모빌리티 예시로 풀어보면:
 - T (Task): 1시간 뒤 강변북로의 교통량 예측
 - E (Experience): 과거 3년치 시간대별 교통량 데이터
 - P (Performance): 예측 오차 (MAE, 평균절대오차)
- 데이터(E)를 더 줄수록 오차(P)가 줄어들면 → "학습하고 있다"

※ T·E·P 정의는 중간고사 단골 문제입니다

딥러닝(DL, Deep Learning)이란?

- 인공신경망(Artificial Neural Network)을 깊게(deep) 쌓은 머신러닝 기법
- 핵심 장점: 특징(feature)을 사람이 설계하지 않고 스스로 추출
 - 전통 ML: 사람이 "엣지, 색상" 등 특징을 설계 → 한계
 - DL: 원시 데이터(픽셀)에서 특징을 계층적으로 자동 학습
- 이미지·음성·자연어처럼 원시 데이터가 복잡한 분야에서 압도적
- 자율주행 인지(카메라 영상 → 객체 인식)가 대표적인 DL 응용
- 대가: 대량의 데이터 + GPU 연산 자원이 필요

규칙 기반 vs 학습 기반 — 코드로 비교

```
# (A) 규칙 기반: 사람이 규칙을 직접 작성
def is_congested_rule(speed_kmh, density):
    """평균속도와 차량밀도로 정체 판단 - 규칙을 내가 정함"""
    if speed_kmh < 30 and density > 80:
        return True
    return False

# (B) 학습 기반: 데이터에서 규칙을 학습 (의사코드)
from sklearn.tree import DecisionTreeClassifier

X = load_traffic_features() # [속도, 밀도, 시간대, 요일, 날씨 ...]
y = load_congestion_labels() # 실제 정체 여부 (과거 기록)

model = DecisionTreeClassifier()
model.fit(X, y) # <-- 규칙을 데이터로부터 '학습'
model.predict([[25, 85, 18, 4, 1]]) # 새로운 상황을 예측
```

※ 규칙 기반은 조건이 5개만 넘어가도 관리 불능 — ML은 데이터가 곧 규칙

머신러닝의 3가지 학습 유형

유형	정답(label)	대표 작업	모빌리티 예시
지도학습 (Supervised)	있음	분류, 회귀	주행 영상 속 보행자 분류, 교통량 예측
비지도학습 (Unsupervised)	없음	군집화, 차원축소	승객 이동 패턴 군집화 (MaaS 노선 설계)
강화학습 (Reinforcement)	보상 신호	순차적 의사결정	신호등 최적 제어, 자율주행 주행 정책

※ 이 수업은 지도학습을 중심으로, 12주차에 생성형 AI를 다룹니다

AI vs ML vs DL 한눈에 비교

구분	AI	머신러닝(ML)	딥러닝(DL)
범위	가장 넓음	AI의 부분집합	ML의 부분집합
방법	규칙, 탐색, 학습 등 모두	데이터로부터 패턴 학습	심층 신경망으로 학습
특징 설계	-	사람이 설계 (feature engineering)	모델이 자동 추출
데이터 요구량	-	중간	대량
모빌리티 예	내비 경로 탐색(A*)	차량 연비 예측(회귀)	카메라 기반 차선 인식

AI는 언제 쓰고, 언제 쓰지 말아야 하나?

AI(ML)가 적합한 경우

- 규칙을 말로 정의하기 어려움 (영상 속 보행자)
- 패턴이 데이터에 숨어 있음 (교통량의 요일 패턴)
- 환경이 변해서 규칙을 계속 갱신해야 함
- 대량의 데이터가 이미 쌓여 있음

전통 방식이 나은 경우

- 규칙이 명확하고 단순함 (요금 계산, 세금)
- 100% 정확성·설명 가능성이 법적으로 요구됨
- 데이터가 거의 없음
- 잘못 판단했을 때 비용이 치명적인데 검증이 불가

■ 중간 점검 퀴즈 #2

Q. 다음 중 옳은 설명은?

- ① 딥러닝은 머신러닝과 별개의 분야다
- ② 머신러닝은 규칙을 사람이 직접 코딩하는 방식이다
- ③ 딥러닝은 특징(feature)을 데이터에서 자동으로 추출한다
- ④ AI라고 부르려면 반드시 신경망을 사용해야 한다

▶ 정답: ③ — AI $\supset$ ML $\supset$ DL, ML은 데이터로 규칙을 학습, 규칙 기반도 AI에 포함

PART 3

AI를 보는 관점 — 결국 함수 찾기

블랙박스, 함수 근사, 그리고 왜 신경망인가

AI 모델은 블랙박스(black-box)입니다

내부를 사람이 읽을 수 있는 규칙으로 설명하기 어렵고, 입력→출력 관계로만 이해합니다

블랙박스란 무슨 뜻인가?

- 전통 프로그램: 코드를 열어 보면 동작 규칙을 한 줄씩 읽을 수 있음
- 학습된 AI 모델: 내부는 수백만~수십억 개의 숫자(가중치) 덩어리
 - "왜 보행자라고 판단했나?"를 규칙으로 답하기 어려움
- 그래서 모델은 입력 → 출력 관계, 즉 하나의 함수로 이해합니다
 - 입력 x (카메라 픽셀) → f → 출력 y ("보행자 있음")
- 내부가 안 보이니, 밖에서의 검증(평가)이 그만큼 중요해짐

함수(函數)라는 말의 어원 — 처음부터 '상자'였다

- function의 동아시아 번역어 = 函數(함수)
 - 한자 函 = '상자'라는 뜻 (예: 우체함, 보관함의 函)
 - 일본도 과거 函数로 표기하다가 이후 関数로 변경 / 한국은 函數에서 온 '함수'를 사용
- 즉 함수라는 말 자체가 "입력을 넣으면 출력이 나오는 상자(box)"
- AI가 블랙박스(black-box)라는 것과 정확히 같은 그림!
 - x를 넣으면 y가 나오는 검은 상자 — 이름부터 운명이었던 셈
- 연결: AI 학습 = 데이터에 맞는 상자(함수)를 찾는 일 — 더 정확히는, 데이터의 확률 분포를 데이터 주도(data-driven)로 맞추는 일

※ 용어의 어원을 알면 개념이 그림으로 기억됩니다

■ 학습(Training)의 본질 = 함수 찾기

- 학습이란: 데이터에 가장 잘 맞는 함수 $f: X \rightarrow Y$ 를 찾는 과정
- 모빌리티 예시:
 - $f(\text{속도, 중량, 도로경사}) = \text{연비}$ — 숫자를 내는 함수 (회귀)
 - $f(\text{카메라 픽셀}) = \text{"보행자 있음/없음"}$ — 범주를 내는 함수 (분류)
- f 는 파라미터(parameter)라는 조절 손잡이를 가진 함수 모임
 - 학습 = 파라미터 공간을 탐색해 데이터와 오차가 최소인 f 를 고르는 것
- "모델을 학습시킨다" = "함수의 손잡이를 데이터에 맞게 돌린다"

장난감 예제: 파라미터를 돌려 함수 찾기

```
# 목표: 차량 중량(x)으로 연비(y)를 맞추는 직선  $f(x) = w \cdot x + b$  찾기
weights = [1.0, 1.2, 1.5, 1.8, 2.0] # 차량 중량 (톤)
mpg      = [18., 16., 13., 10., 9.]  # 실제 연비 (km/L)

def f(x, w, b):          # 파라미터 w, b를 가진 함수 모임
    return w * x + b

def error(w, b):         # 데이터와 얼마나 어긋나는가 (오차)
    return sum((f(x, w, b) - y) ** 2 for x, y in zip(weights, mpg))

# 파라미터 공간을 뒤져서 오차가 가장 작은 (w, b)를 찾는다
best = min(((w/10, b) for w in range(-150, 0) for b in range(0, 40)),
           key=lambda p: error(*p))
print("찾은 함수:  $y = %.1f x + %d$ " % best) # 대략  $y = -9.5x + 28$ 
```

※ 실제 딥러닝은 이 '탐색'을 경사하강법(gradient descent)으로 효율화한 것 — 5주차

확률적 관점: 학습 = 분포(Distribution) 찾기

- 한 단계 더 깊은 시각: 데이터는 어떤 확률 분포에서 나온 표본
- 학습이란 그 데이터셋의 분포를 찾아가는 과정
 - 분류 모델: $p(y|x)$ — "이 입력일 때 각 정답의 확률"을 학습
 - 예: $p(\text{보행자} | \text{이 픽셀들}) = 0.97 \rightarrow \text{"보행자"} \text{로 판단}$
 - 생성 모델: $p(x)$ — "데이터 자체가 나올 확률"을 학습 $\rightarrow$ 새 데이터를 생성 가능
- ChatGPT가 문장을 만드는 것도 $p(\text{다음 단어} | \text{지금까지 문장})$ 의 연속
- 1교시의 그 문장 — "나는 신해철의 '확률'이다" — 이제 수식으로 이해할 수 있다: 고인의 데이터로 $p(x)$ 를 학습, 매회 방송은 그 분포에서 뽑은 새 샘플

※ $p(x)$ 를 배우는 생성형 AI는 12주차에서 다시 만납니다

왜 신경망인가? — 보편 근사 정리 (Universal Approximation Theorem)

- 정리(직관 버전): 은닉층(hidden layer) 하나짜리 신경망도, 뉴런이 충분히 많으면 어떤 연속 함수든 원하는 정확도로 근사할 수 있다
- 비유: 레고 블록이 충분하면 어떤 곡면도 원하는 만큼 매끈하게 흉내 낼 수 있음
 - 뉴런 하나하나는 단순한 꺾인 조각 — 많이 합치면 복잡한 모양이 됨
- 의미: "픽셀 → 보행자" 같은 복잡한 함수도 신경망으로 표현 가능
- 그래서 신경망이 '함수 찾기'의 만능 틀(universal template)로 쓰임

※ 증명은 다루지 않습니다 — '가능하다'는 보장이 있다는 사실만 기억

UAT의 함정 — 정리가 보장하지 않는 것

UAT가 보장하는 것

- 그런 함수가 '존재'한다 (표현 가능성)
- 은닉층 1개 + 충분한 뉴런이면 충분
- 연속 함수라면 무엇이든

UAT가 보장하지 않는 것

- 학습 알고리즘이 그 함수를 실제로 '찾는다'는 보장 (최적화 문제)
- 새 데이터에서도 잘 맞는다는 보장 (일반화, generalization)
- 뉴런이 몇 개면 되는지 (천문학적일 수 있음)
- → 그래서 데이터·최적화·평가가 실력의 전부

블랙박스의 함의: 설명가능 AI(XAI)와 검증

- 블랙박스라서 생기는 문제: "왜?"에 답하지 못하는 AI
 - 자율긴급제동(AEB)이 갑자기 작동 — 원인을 설명 못 하면 신뢰 불가
- XAI (eXplainable AI): 모델의 판단 근거를 해석하려는 연구 분야
 - 어떤 픽셀이 '보행자' 판단에 기여했는지 시각화 등
- 안전 필수(safety-critical) 분야 — 자율주행·의료 — 에서는:
 - 내부 설명이 어려운 만큼, 외부 검증(방대한 시험 주행·시뮬레이션·평가 지표)이 필수
- 이 수업에서 '평가(Evaluation)'를 반복 강조하는 이유가 바로 이것

**학습 = 데이터에 맞는 함수(분포)를
파라미터 공간에서 찾는 일**

이 한 문장을 기억하면 이번 학기의 모든 내용이 이 문장의 각론입니다

PART 4

AI의 역사

튜링 테스트에서 생성형 AI까지 — 70년의 롤러코스터

AI 역사 타임라인 한눈에

시기	사건	키워드
1950	튜링 테스트 제안 (Alan Turing)	"기계가 생각할 수 있는가?"
1956	다트머스 회의 — 'AI' 용어 탄생	1차 AI 봄 시작
1970s	1차 AI 겨울 (AI Winter)	과도한 기대 → 자금 삭감
1980s	전문가 시스템(Expert System) 유행	2차 봄, 이후 2차 겨울
1990s~2000s	머신러닝 부상, 딥블루 체스 승리(1997)	통계적 학습, SVM
2012	AlexNet, ImageNet 대회 압승	딥러닝 혁명의 시작
2016	AlphaGo, 이세돌 9단에 승리	강화학습 + DL
2022~	ChatGPT 등 생성형 AI 대중화	LLM, 파운데이션 모델

1950: 튜링 테스트(Turing Test)

- 앨런 튜링, 논문 "Computing Machinery and Intelligence"
- 질문을 바꿈: "기계가 생각하는가?" → "기계가 사람처럼 대화하는가?"
- 심사자가 채팅만으로 상대가 기계인지 구분 못 하면 통과
 - 지능의 '정의' 대신 '판별 가능한 행동'으로 문제를 치환
- 2020년대 LLM은 짧은 대화에선 사실상 통과 수준
 - 그래서 오늘날엔 더 정교한 벤치마크(benchmark)로 평가

1980s: 전문가 시스템(Expert System)의 흥망

무엇이었나

- 전문가의 지식을 if-then 규칙 수천 개로 코딩
- 의료 진단(MYCIN), 장비 구성(XCON) 등 상용화
- 기업들이 앞다투어 도입 — 2차 AI 붐
- "지식을 넣어주면 기계가 추론한다"는 접근

왜 무너졌나

- 규칙 획득 병목: 전문가 지식을 규칙화하기 너무 어려움
- 유지보수 지옥: 규칙 수천 개가 서로 충돌
- 예외 상황에 극도로 취약 (상식 부재)
- → 2차 AI 겨울, 그리고 '학습' 기반으로 방향 전환

1990s~2000s: 머신러닝의 부상

- 발상의 전환: 지식을 '넣어주지' 말고 데이터에서 '배우게' 하자
- 통계적 학습 이론 성숙: 결정트리, SVM, 랜덤포레스트
- 1997년 딥블루(Deep Blue), 체스 세계 챔피언 카스파로프 격파
- 인터넷 보급 → 학습에 쓸 데이터가 폭증하기 시작
- 스팸 필터, 추천 시스템 등 조용하지만 확실한 상용 성공
 - 이 시기의 ML이 지금도 정형 데이터 분석의 주력

2012: 딥러닝 혁명 — AlexNet

- ImageNet 이미지 인식 대회: 오차율 26% → 16%로 단숨에 단축
- 성공의 3박자가 마침내 갖춰짐:
 - ① 빅데이터: 라벨링된 이미지 120만 장 (ImageNet)
 - ② GPU: 게임용 그래픽카드로 병렬 연산 (NVIDIA)
 - ③ 알고리즘: ReLU, Dropout 등 학습 기법 개선
- 이후 이미지 인식은 인간 수준 초과 → 자율주행 인지 기술의 토대
- 2016년 AlphaGo가 대중에게 각인시킨 딥러닝의 위력

2022~: 생성형 AI(Generative AI) 시대

- 판별(discriminative)을 넘어 생성(generative)으로
 - 기존: 이 사진이 고양이인가? → 이제: 고양이 사진을 그려줘
- LLM(대형 언어 모델): 방대한 텍스트로 사전학습된 범용 모델
- ChatGPT(2022) — 역사상 가장 빠르게 사용자 1억 명 도달
- 코드 작성, 문서 요약, 이미지 생성이 일상 도구화
- 모빌리티에서도: 차량 음성 비서, 주행 시나리오 생성, 관제 보고서 자동화

※ 12주차에 생성형 AI와 LLM 활용을 실습합니다

나란히 보기: AI 역사 vs 자율주행 역사

AI 이정표

- 1950 튜링 테스트
- 1956 다트머스 회의
- 1997 딥블루 체스 승리
- 2012 AlexNet (딥러닝 혁명)
- 2016 AlphaGo
- 2022 ChatGPT

자율주행 이정표

- 1995 CMU NavLab 'No Hands Across America'
- 2004~05 DARPA 그랜드 챌린지 (사막 자율주행)
- 2007 DARPA 어반 챌린지 (도심)
- 2014 Tesla Autopilot 발표
- 2018~ Waymo 상용 로보택시 개시
- 2020s 국내 레벨3 양산차 등장

**AI의 역사 = 규칙에서 학습으로,
그리고 판별에서 생성으로**

봄과 겨울의 반복 속에서 살아남은 것은 '데이터와 컴퓨팅'이라는 토대였습니다

PART 5

스마트모빌리티 × AI

차 한 대, 도시 하나에 들어가는 AI를 모두 세어 보자

모빌리티는 AI의 최전선입니다

- 차량 1대 = 카메라·레이더·LiDAR 등 수십 개 센서의 데이터 공장
 - 자율주행차 1대가 하루 수 TB의 데이터를 생성
- 차량 안: 인지·판단·제어, ADAS, 예지정비, 음성 비서
- 도로 위: 교통량 예측, 신호 최적화, V2X 통신
- 서비스: MaaS, 수요응답형 교통(DRT), 라이드헤일링 배차
- 이 과목의 모든 실습 예제가 여기서 나옵니다

사례 1: 자율주행 인지(Perception)

- 자율주행 3단계: 인지(Perception) → 판단(Planning) → 제어(Control)
- 인지 = 센서 데이터로 주변 세계를 이해하는 것, DL의 주 무대
 - 객체 탐지(Object Detection): 차량·보행자·자전거 위치 찾기 → 8주차
 - 차선 인식·세그멘테이션(Segmentation): 픽셀 단위 도로 이해 → 10주차
 - 센서 융합(Sensor Fusion): 카메라 + LiDAR + 레이더 결합
- 악천후·역광·비정형 상황(공사장)이 여전히 난제

자율주행 레벨 (SAE J3016)

레벨	이름	운전 주체	예시
0	비자동화	인간	일반 차량
1	운전자 보조	인간	어댑티브 크루즈 or 차선유지 중 하나
2	부분 자동화	인간 (상시 감시)	고속도로 주행 보조 (양손은 핸들에)
3	조건부 자동화	시스템 (조건부)	정체 구간 자율주행, 요청 시 인간 개입
4	고도 자동화	시스템 (한정 구역)	로보택시 (지정 지역 내 무인)
5	완전 자동화	시스템 (모든 조건)	핸들 없는 차 (아직 없음)

※ 레벨 2와 3 사이가 기술적·법적으로 가장 큰 간극

사례 2: ADAS (첨단 운전자 보조 시스템)

- **ADAS = Advanced Driver Assistance Systems, 레벨 1~2의 핵심**
 - AEB (자동긴급제동): 전방 충돌 위험 감지 시 자동 제동
 - LKA (차선유지보조): 카메라로 차선 인식 후 조향 보조
 - SCC (스마트크루즈): 앞차와의 거리 유지하며 속도 조절
 - BSD (후측방 경고): 사각지대 차량 감지
- **모두 실시간 AI 인지 모델이 차량 내 칩에서 동작 (엣지 AI)**
- **신차 안전 평가(NCAP)에서 ADAS가 필수 항목이 되며 급성장**

사례 3: 교통량 예측 (Traffic Forecasting)

- 과거 교통 데이터로 미래 30분~수 시간의 교통량·속도를 예측
- 데이터: 루프 검지기, GPS 궤적, 하이패스, 내비게이션 로그
- 패턴: 출퇴근 이중 피크, 요일 효과, 날씨·이벤트 영향
 - 시계열(Time Series) 모델의 대표 응용 → 11주차 RNN에서 실습
- 활용: 내비 도착시간(ETA), 신호 제어, 교통정보 제공
- 오늘 시연에서 '가상의 시간대별 교통량' 그래프를 직접 그려봅니다

사례 4: MaaS (Mobility as a Service)

- 여러 교통수단(버스·지하철·킵보드·택시)을 하나의 앱으로 통합
- AI의 역할:
 - 최적 경로 조합: 멀티모달 경로 탐색 + 개인 선호 학습
 - 수요 예측: 시간·지역별 이동 수요를 예측해 공급 배치
 - DRT(수요응답형 교통): 실시간 승차 요청을 묶어 동적 노선 생성
- 국내 사례: 세종·인천 등 DRT 셔틀, 통합 교통 앱 실증 사업
- 데이터 과학 + 최적화 + ML이 결합된 종합 문제

■ 사례 5: 차량 예지정비 (Predictive Maintenance)

- 고장 나면 고치는 것(사후)이 아니라, 고장 나기 전에 예측(예지)
- 데이터: 엔진 온도, 진동, 배터리 전압, OBD 진단 코드, 주행 이력
- 모델: 이상 탐지(Anomaly Detection) + 잔여수명(RUL) 예측
- 효과: 버스·트럭 플릿(fleet) 운영사의 정비 비용·다운타임 절감
 - 전기차 배터리 열화 예측은 특히 뜨거운 주제
- 센서 시계열 데이터 분석 — 11주차 내용과 직결

사례 6: V2X (Vehicle-to-Everything)

- 차량이 차량(V2V)·인프라(V2I)·보행자(V2P)·네트워크(V2N)와 통신
- 센서가 못 보는 곳을 통신으로 보완: "교차로 사각지대의 차량 정보 수신"
- AI의 역할:
 - 수신 데이터 + 자차 센서의 융합 판단 (협력 인지)
 - 교차로 신호 최적화: 강화학습 기반 실시간 신호 제어 연구 활발
- C-ITS 실증: 국내 고속도로·도심에서 진행 중

스마트모빌리티 AI 사례 총정리

분야	핵심 AI 기술	데이터	이 수업 주차
자율주행 인지	CNN, 객체탐지, 세그멘테이션	카메라 영상, LiDAR	7~10주차
ADAS	실시간 엣지 추론	카메라, 레이더	8주차
교통량 예측	시계열 모델 (RNN 등)	검지기, GPS 궤적	11주차
MaaS / DRT	수요예측 + 최적화	승하차 로그, 이동 패턴	13주차
예지정비	이상탐지, 회귀	차량 센서 시계열	11·13주차
V2X / 신호제어	강화학습, 융합 판단	통신 메시지, 신호 데이터	13주차

※ 이 표가 사실상 한 학기 커리큘럼 지도입니다

모빌리티 산업의 경쟁력은 하드웨어에서 소프트웨어와 AI로 이동 중

SDV(Software Defined Vehicle) 시대 — 여러분이 배울 기술이 곧 산업의 언어입니다

■ 중간 점검 퀴즈 #3

Q. 다음 상황에 가장 알맞은 AI 응용 분야는? "시내버스 운영사가 고장으로 인한 운행 중단을 줄이고 싶다"

- ① 자율주행 인지 (Perception)
- ② 차량 예지정비 (Predictive Maintenance)
- ③ MaaS 통합 경로 탐색
- ④ V2X 협력 인지

▶ 정답: ② — 센서 시계열로 고장 징후를 미리 탐지해 정비 시점을 앞당기는 문제

PART 6

AI 개발 파이프라인과 Google Colab

데이터 → 모델 → 평가 → 배포, 그리고 우리의 실습 환경

AI 개발 파이프라인 4단계

- ① 데이터 (Data): 수집 → 정제 → 라벨링 → 분할
- ② 모델 (Model): 알고리즘 선택 → 학습(training)
- ③ 평가 (Evaluation): 시험 데이터로 성능 측정 → 부족하면 ①②로 회귀
- ④ 배포 (Deployment): 실제 서비스·차량에 탑재, 모니터링
- 직선이 아니라 순환(loop): 배포 후에도 데이터를 모아 다시 개선
 - 실무 시간의 60~80%는 ① 데이터 단계에 쓰인다는 말이 있을 정도

① 데이터 단계 — 모빌리티 예시로

- 수집: 시험 차량이 주행하며 카메라 영상 기록 (하루 수 TB)
- 정제: 흐린 프레임 제거, 개인정보(얼굴·번호판) 비식별화
- 라벨링(Labeling): 사람이 영상 속 차량·보행자에 박스를 그림
 - 라벨링 비용이 자율주행 개발비의 큰 몫 — 자동 라벨링 연구 활발
- 분할: 학습(train) / 검증(validation) / 시험(test) 세트로 나눔
 - 시험 세트는 '수능 문제지' — 학습에 절대 사용 금지

② 모델 학습과 ③ 평가

모델 학습 (Training)

- 문제 유형에 맞는 알고리즘 선택
- 학습 = 데이터에 맞게 모델 내부 숫자(가중치)를 조정
- GPU로 수 시간~수 주 소요 (규모에 따라)
- 하이퍼파라미터 튜닝: 학습 설정값 탐색

평가 (Evaluation)

- 학습에 안 쓴 시험 데이터로 측정해야 공정
- 지표: 정확도, 정밀도/재현율, MAE 등 (4주차)
- 보행자 탐지는 '놓침(미탐)'이 치명적 → 재현율 중시
- 기준 미달이면 데이터·모델 단계로 되돌아감

④ 배포(Deployment) — 모델을 세상으로

- 서버 배포: 교통량 예측 API처럼 클라우드에서 서비스
- 엣지 배포: ADAS처럼 차량 내 칩에서 실시간 추론
 - 차량 탑재는 경량화(모델 압축)와 실시간성이 관건
- OTA(Over-The-Air): 무선 업데이트로 차량의 AI를 갱신 (SDV 핵심)
- 모니터링: 배포 후 성능 저하 감시 (데이터 분포 변화 = drift)
- 이 수업에서는 ①~③을 집중 실습, ④는 프로젝트에서 개념 체험

파이프라인 단계별 산출물과 도구

단계	산출물	대표 도구	이 수업
데이터	정제된 데이터셋, 라벨	pandas, NumPy	2주차~
모델	학습된 모델 파일	scikit-learn, PyTorch	3~6주차
평가	성능 리포트, 지표	scikit-learn, matplotlib	4주차~
배포	API/웹지 모듈	(개념만) ONNX, 클라우드	14~15주차

Google Colab — 우리의 실습 환경

- 브라우저에서 실행되는 무료 Jupyter 노트북 (구글 계정만 필요)
- 설치 제로: Python, NumPy, PyTorch가 이미 깔려 있음
- 무료 GPU 제공: 딥러닝 실습(6주차~)에 필수
- 노트북 = 코드 셀 + 마크다운(설명) 셀의 조합 문서
- 자동으로 Google Drive에 저장 → 과제 파일 관리 용이
 - colab.research.google.com 접속 → 새 노트북

Colab 필수 조작법

작업	방법	비고
셀 실행	Shift + Enter	실행 후 다음 셀로 이동
셀 실행(제자리)	Ctrl + Enter	커서가 그대로
코드 셀 추가	상단 [+ 코드] 버튼	단축키: Ctrl+M B (아래에)
마크다운 셀 추가	상단 [+ 텍스트] 버튼	설명·제목 작성용
파일 업로드	왼쪽 폴더 아이콘 → 업로드	런타임 초기화 시 사라짐!
GPU 켜기	런타임 → 런타임 유형 변경 → GPU	6주차부터 사용

※ 오늘 시연에서 전부 직접 해봅니다

Colab 주의사항 (과제 제출 전 필독)

- 세션 제한: 일정 시간 방치하면 런타임이 끊기고 변수가 사라짐
 - 끊기면: 런타임 → 모두 실행 으로 처음부터 재실행하면 됨
- 업로드 파일은 세션 종료 시 삭제 → 중요한 파일은 Drive에 마운트
- 과제 제출 전: "런타임 다시 시작 후 모두 실행"으로 처음부터 끝까지 에러 없이 도는지 확인
- 무료 GPU는 사용량 제한 있음 — 필요할 때만 켜기
- 다운로드: 파일 → 다운로드 → .ipynb 로 받아 GitHub push(또는 이메일 첨부) + e-Campus 제출

여러분의 첫 파이썬 셀

```
# Colab에서 이 셀을 만들어 Shift+Enter로 실행해 보세요
```

```
print("Hello, Smart Mobility!")
```

```
# 변수와 계산
```

```
distance_km = 12.5      # 통학 거리
```

```
speed_kmh = 25           # 평균 속도
```

```
time_min = distance_km / speed_kmh * 60
```

```
print(f"통학 예상 시간: {time_min:.0f}분")
```

```
# 리스트와 반복문 맛보기
```

```
sensors = ["카메라", "레이더", "LiDAR", "GPS"]
```

```
for s in sensors:
```

```
    print(f"{s} 센서: 정상 작동")
```

※ 출력이 나오면 성공 — 오늘 시연 노트북의 첫 번째 셀입니다

[시연] Week01_demo.ipynb 따라하기

- 지금부터 30분: 교수 시연을 보며 같은 노트북을 직접 실행
- 노트북 열기: Google Drive 공유 폴더에서 노트북 열기 → Colab에서 열림 → 내 Drive에 사본 저장 (공유 폴더 링크는 Mattermost 채널 공지 참조)
- 구성:
 - ① Colab 기본 조작: 셀 실행, 마크다운, 파일 업로드
 - ② Python 빠른 점검: 변수, 리스트, 함수, for문
 - ③ 시각화 맛보기: 시간대별 교통량 그래프 그리기
- 막히면 옆 사람과 화면을 비교하세요 (피어 튜터링 예행연습)

[시연 미리보기] 시간대별 교통량 그래프

```
import numpy as np
import matplotlib.pyplot as plt

hours = np.arange(24)          # 0시 ~ 23시
base = 800 + 900 * np.exp(-((hours - 8) ** 2) / 4) \
      + 1100 * np.exp(-((hours - 18) ** 2) / 5)
traffic = base + np.random.normal(0, 60, 24)  # 노이즈 추가

plt.figure(figsize=(9, 4))
plt.plot(hours, traffic, color="#2563A8", linewidth=2, marker="o")
plt.title("Hourly Traffic Volume (Synthetic)")
plt.xlabel("Hour of Day")
plt.ylabel("Vehicles / hour")
plt.grid(alpha=0.3)
plt.show()
```

※ 출퇴근 이중 피크(8시, 18시)가 보이면 성공 — 코드 의미는 2주차부터 차근차근

오늘의 정리

- ✓ 이 수업: 시연 중심 + 자동채점 빈칸 과제(15%) + 3인 1조 피어 튜터링
- ✓ 평가: 중간 25 / 기말 25 / 프로젝트 25 / 과제 15 / 출석 10
- ✓ $AI \supset ML \supset DL$ — 머신러닝은 데이터로 규칙을 학습하는 방법
- ✓ 학습의 본질 = 데이터에 맞는 함수(분포)를 파라미터 공간에서 찾기 (UAT: 가능성 보장, 학습·일반화는 별개)
- ✓ AI 역사: 튜링 테스트 → 전문가 시스템 → ML → 딥러닝 → 생성형 AI
- ✓ 모빌리티 AI: 인지, ADAS, 교통량 예측, MaaS, 예지정비, V2X
- ✓ 파이프라인: 데이터 → 모델 → 평가 → 배포 (그리고 순환)

다음 주: 2주차 — Python 데이터 처리 기초 (변수·자료구조·NumPy·pandas 입문)

다음 주 예고 & 오늘의 할 일

- 2주차: Python 데이터 처리 기초

- 변수·리스트·딕셔너리·함수 — 오늘 맛본 것을 제대로
- NumPy 배열과 pandas 데이터프레임으로 교통 데이터 다루기

- 오늘의 할 일 (다음 수업 전까지):

- ① Google 계정으로 Colab 접속 확인 (5분)
- ② Week01_demo.ipynb 를 처음부터 끝까지 스스로 재실행 (과제 아님, 워밍업)
- ③ Mattermost 채널의 설문 링크로 조 편성 설문 제출 (학년·프로그래밍 경험 조사)